

Recap: Berechenbarkeit II

- Satz von Rice:
 - Sei S eine nichtleere echte Teilmenge aller berechenbaren Funktionen, dann gilt $L(S) = \{\langle M \rangle \mid M \text{ berechnet eine Funktion aus } S\}$ ist nicht entscheidbar
 - nicht-triviale Eigenschaften von Algorithmen sind nicht entscheidbar

Komplexitätstheorie

Ziele der Komplexitätstheorie

- Die Komplexitätstheorie beschäftigt sich mit der "Komplexität von Problemen".
- Definiert durch die benötigten Ressourcen für die Problemlösung
 - Rechenzeit, Speicherbedarf
- Das ist anders als die (aber verbunden mit der) Komplexität von Algorithmen.
 - Komplexität eines Algorithmus gibt uns obere Schranke, z.B. $O(n^2)$, $O(n \cdot \log n)$
 - Gesucht: untere Schranken
- Welche Probleme können durch "Polynomialzeitalgorithmen" gelöst werden?

O-Notation

- Es gibt einen engen Zusammenhang zur sogenannten **O-Notation**:

Seien $f, g : \mathbb{N} \rightarrow \mathbb{N}$ Funktionen. Wir sagen, dass f **höchstens so schnell wächst** wie g , falls folgendes gilt:

Es gibt eine Konstante $C \in \mathbb{N}$ und ein $N \in \mathbb{N}$, so dass für jedes $n \geq N$ gilt:

$$f(n) \leq C \cdot g(n)$$

- **Anschaulich**: ab einem bestimmten Wert N ist f kleiner als g , multipliziert mit einem konstanten Faktor. (Nur das **asymptotische Verhalten** zählt, Konstanten werden vernachlässigt.)

O-Notation

- **Schreibweisen:**

- Eine Funktion f wird hier oft durch ihren definierenden Ausdruck beschrieben: $n, n^3, 2^n, \dots$
- Dabei wird im Allgemeinen n als Funktionsparameter verwendet.
- $O(g)$ bezeichnet die Menge aller Funktionen, die höchstens so schnell wachsen wie g . Man schreibt dann $f \in O(g)$ (oder sogar $f = O(g)$).

- **Beispiele:**

- | | |
|--------------------|------------------|
| • $n \in O(n^2)$ | $100n \in O(n)$ |
| • $n^2 \in O(2^n)$ | $n^3 \in O(2^n)$ |

- **Bemerkung:** Für das letzte Beispiel gilt $n^3 < 2^n$, falls $n \geq 10$.

O-Notation

- Polynomiales vs. exponentielles Wachstum
- Seien $m \in \mathbb{N}$, $q \in \mathbb{R}$ mit $q > 1$. Dann gilt

$$n^m \in O(q^n)$$

- Das bedeutet, dass jede **Exponentialfunktion** (mit Basis größer als 1) mindestens so stark wächst wie jedes **Polynom**.

(Exponentialfunktionen wachsen sogar echt stärker als Polynome.)

O-Notation

- Man spricht von
 - linearer Laufzeit: $O(n)$
 - quadratischer Laufzeit: $O(n^2)$
 - kubischer Laufzeit: $O(n^3)$
 - polynomialer Laufzeit: $O(n^m)$ für eine Konstante m bzw. $O(p(n))$ für ein Polynom p
 - exponentieller Laufzeit: $O(2^n)$ bzw. $O(q^n)$ für $q > 1$

Laufzeit

- **worst case Laufzeit** $t_A(n)$ eines Algorithmus A
 - maximale Laufzeitkosten auf Eingaben der Länge n bezüglich des logarithmischen Kostenmaßes der RAM
- Hier: Immer (worst case) Laufzeit, auch wenn wir den Zusatz *worst case* nicht explizit erwähnen.
- Wir sagen, die worst case Laufzeit $t_A(n)$ eines Algorithmus A ist **polynomiell beschränkt**, falls gilt $\exists \alpha \in \mathbb{N}: t_A(n) \in O(n^\alpha)$
- **Polynomialzeitalgorithmus** : Algorithmus mit polynomiell beschränkter worst case Laufzeit

Laufzeit

Viele Ihnen bekannte Algorithmen sind Polynomialzeitalgorithmen:

- Sortieralgorithmen
 - Quicksort, Heapsort, Mergesort (Timsort), Insertionsort, Bubblesort, Selectionsort, Bucketsort
- Algorithmen in Baumstrukturen
 - RS-Bäume, AVL-Bäume, B-Bäume, B*-Bäume (Einfügen, Finden, Löschen, Aufbauen)
- Einfach und doppelt verkettete Listen (Einfügen, Finden, Löschen, Aufbauen)
- Kürzeste Wegesuche in Graphen
 - Van Dijkstra-Algorithmus
- Algorithmen zum Suchen von minimalen Spannbäumen
 - Algorithmus von Prim

Laufzeit

- Turingmaschine (TM) und RAM können sich gegenseitig mit polynomiell Zeitverlust simulieren. Wenn ein Problem also einen Algorithmus mit polynomiell beschränkter Laufzeit auf der RAM hat, so kann es auch mit einer polynomiellen Anzahl Schritten auf einer TM gelöst werden, und umgekehrt.
 - Auch für zahlreiche andere sinnvolle Maschinenmodelle, wie Mehrband-TM.
- Die Klasse der Probleme, die in polynomieller Zeit gelöst werden, ist also in einem gewissen Sinne robust gegenüber der Wahl des Maschinenmodells. (Potentielle Ausnahme: Quantencomputer)

Komplexitätsklasse P

- **P** ist die Klasse der Probleme, für die es einen Polynomialzeitalgorithmus gibt.
- Polynomialzeitalgorithmen werden häufig (stark vereinfacht) auch als **”effiziente Algorithmen”** bezeichnet. P ist in diesem Sinne die Klasse derjenigen Probleme, die effizient gelöst werden können.
- Dies ist eine Teilmenge der berechenbaren/rekursiven Probleme.
- Hinweis
 - P besteht nicht aus Algorithmen (sondern Problemen)!
 - Oft werden andere Klassen (wie BPP) als diejenigen Probleme bezeichnet, die effizient gelöst werden können.

Komplexitätsklasse P

- Damit ist es nun auch möglich, die Klasse aller Sprachen zu definieren, die von deterministischen Turingmaschinen mit **polynomialer Laufzeitbeschränkung** erkannt werden.

$$\begin{aligned}
 P &= \{A \mid \text{es gibt eine det. Turingmaschine } M \text{ und ein Polynom } p \text{ mit } T(M) = A \text{ und} \\
 &\quad \text{time}_M(x) \leq p(|x|)\} \\
 &= \bigcup_{p \text{ Polynom}} \text{TIME}(p(n))
 \end{aligned}$$

- Intuitiv umfasst P alle Probleme, für die effiziente Algorithmen existieren.

Komplexitätsklasse P

- Zusammenhang zwischen P und der O-Notation
 - Ein Problem liegt in P genau dann, wenn es von einer (deterministischen) Maschine gelöst werden kann, die polynomiell viele Schritte macht, d.h. $O(n^R)$ Schritte für eine Konstante f , falls n die Eingabegröße ist.
- Komplexitätsklassen für andere Berechnungsmodelle
 - Die Klasse P ist größtenteils unabhängig von dem betrachteten Maschinenmodell.
 - Wir haben gesehen, dass eine Simulation zwischen RAM und TM in polynomieller Zeit möglich ist
 - Die Voraussetzung hierfür ist jedoch, dass das logarithmische Kostenmaß (siehe nächste Folie) angewandt wird.

Komplexitätsklasse P

Beispiele

- Kürzeste Wege
- Minimaler Spannbaum
- Graphzusammenhang
- Maximaler Fluss
- Maximum Matching
- Lineare Programmierung
- Größter Gemeinsamer Teiler
- Primzahltest (nicht-trivial: AKS-test)

Komplexitätsklasse P

Problem (**Sortieren**)

- Eingabe: N (binär kodierte) Zahlen $a_1, \dots, a_N \in \mathbb{N}$
- Ausgabe: aufsteigend sortierte Folge der Eingabezahlen

Satz: Sortieren $\in P$.

Beweis:

Sei n die Eingabelänge.

Im uniformen Kostenmaß können wir in Zeit $O(N \log(N))$ sortieren, z.B. durch Mergesort. Wir zeigen, die Laufzeit von Mergesort ist auch bezüglich des logarithmischen Kostenmaßes polynomiell beschränkt.

Jeder uniforme Schritt von Mergesort kann in Zeit $O(l)$ durchgeführt werden, wobei $l = \max_i \log(a_i)$. Die Laufzeit von Mergesort ist somit $O(l \cdot N \cdot \log(N))$

Aus $l \leq n$ und $N \leq n$, folgt $l \cdot N \cdot \log(N) \leq n^2 \log(n) \leq n^3$. Damit ist die worst case Laufzeit von Mergesort polynomiell beschränkt durch $O(n^3)$.

QED

Komplexitätsklassen

- Um nachzuweisen, dass ein Problem in P enthalten ist, genügt es einen Polynomialzeitalgorithmus für das Problem anzugeben.
 - “einfach”
 - Das haben Sie in “Algorithmen und Datenstrukturen” gelernt.
- Wie kann man aber nachweisen, dass ein Problem nicht in P enthalten ist? Diese Frage wird uns noch ein wenig beschäftigen.
- Dazu machen wir einen vielleicht zunächst etwas obskur erscheinenden Umweg über nichtdeterministische Turingmaschinen.

Nichtdeterministische Turingmaschine

- Eine **nichtdeterministische Turingmaschine (NTM)** ist definiert wie eine deterministische Turingmaschine (TM), nur die Zustandsüberföhrungsfunktion wird zu einer Relation $\delta \subseteq (Q \setminus \{\bar{q}\} \times \Gamma) \times (Q \times \Gamma \times \{L, R, N\})$
- Eine Konfiguration K' ist **direkter Nachfolger** einer Konfiguration K , $K \vdash K'$, falls k' durch einen der in δ beschriebenen Übergänge aus K hervorgeht.
 - Zu einer Konfiguration kann es mehrere Nachfolgekonfigurationen geben!
- Jede mögliche Konfigurationsfolge, die mit der Startkonfiguration beginnt und jeweils mit einer der direkten Nachfolgekonfigurationen fortgesetzt wird bis sie eine Endkonfiguration im Zustand $\bar{q}$ erreicht, wird als **Rechenweg** der NTM bezeichnet.
 - Der Verlauf der Rechnung ist nicht eindeutig bestimmt!

Nichtdeterministische Turingmaschine

- Eine NTM M **akzeptiert** die Eingabe $x \in \Sigma^*$, falls es mindestens einen Rechenweg von M gibt, der in eine akzeptierende Endkonfiguration führt. Die von M **erkannte Sprache** $L(M)$ besteht aus allen von M akzeptierten Wörtern.
- Wir behaupten nicht, dass man eine derartige Maschine bauen kann.
- Wir benutzen die NTM nur als Modell für die Klassifikation der Komplexität von Problemen.
 - Und ja, NTM's sind (vermutlich) nicht äquivalent zu Quantencomputern.

Nichtdeterministische Turingmaschine

- Für Laufzeitkosten der NTM M sind nur Eingaben $x \in L(M)$ relevant. Für jedes $x \in L(M)$ gibt es mindestens einen Rechenweg auf dem M zu einem akzeptierendem Endzustand gelangt.
 - Vorstellung: M wählt den kürzesten dieser Rechenwege.
 - Vorstellung: M hat die magische Fähigkeit den kürzesten Rechenweg zu raten.
 - Vorstellung: M kann allen Rechenwegen gleichzeitig folgen und stoppt, sobald ein Weg erfolgreich ist.
- Sei M eine NTM. Die **Laufzeit** von M auf einer Eingabe $x \in L(M)$ ist definiert als $T_M(x)$, die Länge des kürzesten akzeptierenden Rechenweges von M auf x . Für $x \notin L(M)$ definieren wir $T_M(x) = 0$.
- Die (**worst case**) **Laufzeit** $t_M(n)$ für M auf Eingaben der Länge $n \in \mathbb{N}$ ist definiert durch $t_M(n) := \max\{T_M(x) \mid x \in \Sigma^n\}$.

NTIME

- Für die Laufzeit einer NTM M schreiben wir auch $ntime_M$
- **Bemerkung:**
 Falls eine Turingmaschine M gegeben ist, die $ntime_M(x) \leq f(|x|)$ für alle Wörter x erfüllt, so kann man einen Zähler mitlaufen lassen und damit sicherstellen, dass die Maschine immer spätestens nach $f(|x|)$ (ursprünglichen) Schritten anhält.
- Denn ab diesem Schritt muss man einen akzeptierenden Zustand gefunden haben, wenn x in der Sprache von M liegt.

NTIME

- **NTIME**(f) ist die Menge der Sprachen, die von einer nicht-deterministischen Turingmaschine in Zeit $O(f)$ akzeptiert werden können.
- Daraus folgt, dass $\text{NTIME}(f)$ nur entscheidbare Sprachen enthält und man auch bei einer nicht-akzeptierenden Berechnung nicht mehr als $f(|x|)$ Schritte machen muss (plus den im Wesentlichen vernachlässigbaren Overhead für den Zähler).

Komplexitätsklasse NP

- NP ist die Klasse der Entscheidungsprobleme, die durch eine NTM M erkannt werden, deren worst case Laufzeit $t_M(n)$ polynomiell beschränkt ist.
- $NP := \bigcup_{k=0}^{\infty} NTIME(n^k)$
- NP steht dabei für **nichtdeterministisch polynomiell**.
- NP bedeutet **NICHT** “Nichtpolynomiell”

CLIQUE

Problem (**Cliquenproblem** – **CLIQUE**)

- Eingabe: Graph $G = (V, E)$, $k \in \{1, \dots, |V|\}$
- Frage: Gibt es eine k -Clique?

Eine **Clique** ist dabei eine Knotenteilmenge $C \subseteq V$, die die Bedingung erfüllt, dass jeder Knoten aus C mit jedem anderen Knoten aus C durch eine Kante verbunden ist.

Eine **k -Clique** ist eine Clique der Größe k .

Der Graph $G = (V, E)$ sei durch Adjazenzmatrix beschrieben.

Satz: CLIQUE $\in$ NP.

CLIQUE

Beweis

Beschreiben NTM M mit $L(M) = \text{CLIQUE}$

- Falls die Eingabe nicht von der Form (G, k) ist, so verwirft M die Eingabe. Sonst weiter.
- Sei $G = (V, E)$ und N die Anzahl der Knoten. OBdA $V = \{1, \dots, N\}$.
- M schreibt hinter die Eingabe den String $\#^N$.
- Der Kopf bewegt sich unter das erste $\#$.
- (*) M läuft von links nach rechts über den String $\#^N$ und ersetzt ihn nichtdeterministisch durch einen String aus $\{0, 1\}^N$. Diesen String nennen wir $(y_1, \dots, y_N)$.
- Sei $C = \{i \in V \mid y_i = 1\} \subseteq V$.
- M akzeptiert, falls C eine k -Clique ist.

CLIQUE

Beweis (cont.)

Alle Anweisungen außer (*) sind deterministisch, können offensichtlich von (deterministischer) TM in polynomiell beschränkter Zahl von Schritten ausgeführt werden.

Nur in Anweisung (*) greifen wir auf Nichtdeterminismus zurück: M "rät" einen String $y \in \{0,1\}^N$ in N nicht-deterministischen Schritten.

Wir zeigen $L(M) = \text{CLIQUE}$.

Zunächst nehmen wir an, die Eingabe ist von der Form (G, k) und G enthält eine k –CLIQUE. In diesem Fall gibt es mindestens ein y , so dass die Eingabe akzeptiert wird.

Falls die Eingabe hingegen nicht in CLIQUE enthalten ist, so gibt es keinen akzeptierenden Rechenweg.

Fazit: M erkennt die Sprache CLIQUE und hat eine polynomiell beschränkte Laufzeit. Daraus folgt die Behauptung.

QED

Zertifikate

- Im Beweis: Nichtdeterminismus nur zum Raten des Strings y . Wir können y als ein **Zertifikat** ansehen, so dass wir mit diesem Zertifikat die Zugehörigkeit zu CLIQUE in polynomieller Zeit überprüfen können.
- Schwierigkeit des Cliquenproblems: Finden eines geeigneten Zertifikat.
- Verallgemeinere Zertifikate auf alle Probleme in NP, so dass man ohne NTMs auskommt: Eine Sprache L gehört genau dann zu NP, wenn es für die Eingaben aus L ein Zertifikat polynomieller Länge gibt, mit dem sich die Zugehörigkeit zu L in polynomieller Zeit verifizieren lässt.

Zertifikate

Satz: Eine Sprache $L \subseteq \Sigma^*$ ist genau dann in NP, wenn es einen Polynomialzeitalgorithmus V (einen sogenannten **Verifizierer**) und ein Polynom p mit der folgenden Eigenschaft gibt:

$$x \in L \Leftrightarrow \exists y \in \{0,1\}^*, |y| \leq p(|x|): V \text{ akzeptiert } y\#x$$

Beweis:

“genau dann”: es sind 2 Richtungen zu zeigen.

Zertifikate

Beweis (cont.):

“ $\Rightarrow$ ”:

- Sei $L \in \text{NP}$ und M eine NTM, die L in polynomieller Zeit erkennt.
- Die Laufzeit von M sei nach oben beschränkt durch ein Polynom q .
- Konstruiere Verifizierer V mit den gewünschten Eigenschaften.
- OBdA: Die Überführungsrelation δ hat immer genau zwei mögliche Übergänge, mit 0 und 1 bezeichnet.
- Bei Eingabe $x \in \Sigma^n$ verwende Zertifikat $y \in \{0,1\}^{q(n)}$.
- Der Verifizierer V erhält als Eingabe $y\#x$ und simuliert einen Rechenweg der NTM M für die Eingabe x , wobei er im i -ten Rechenschritt von M den Übergang y_i wählt.
- Es gilt $x \in L \Leftrightarrow M \text{ akzeptiert } x \Leftrightarrow \exists y \in \{0,1\}^{q(n)}: V \text{ akzeptiert } y\#x$.
- Laufzeit von V ist polynomiell beschränkt.
- Somit erfüllt V die im Satz geforderten Eigenschaften.

Zertifikate

Beweis (cont.):

„“

- Verifizierer V für L und Polynom p , wie im Satz beschrieben.
- Zeige: $L \in \text{NP}$. Konstruiere dazu NTM M , die L in polynomieller Zeit erkennt.
 - M rät Zertifikat $y \in \{0,1\}^*$, $|y| \leq p(n)$.
 - M führt V auf $y\#x$ aus und akzeptiert, falls V akzeptiert.
- Die Laufzeit von M ist polynomiell beschränkt, und M erkennt L , denn

$$x \in L \Leftrightarrow \exists y \in \{0,1\}^*, |y| \leq p(n): V \text{ akzeptiert } y\#x \Leftrightarrow M \text{ akzeptiert } x$$

QED